When Does Speculative Decoding Stop Paying Off?

A Controlled Study of Context Length on an 8GB GPU

DSAI 4207 Introduction to Large Language Models

Course Project Proposal

Project and Team Details

Team members	Fang Chuming (23097386D); Deng Xiyue (23110509D); Cheng Shuyue (23097089D)
Main contact	Fang Chuming; 23097386d@connect.polyu.hk
Project type	(E) Efficiency & deployment
One-line focus	How context length affects speculative-decoding speedup under a fixed model pair, precision and configuration.

1 Problem Statement and Objective

Speculative decoding uses a smaller draft model to propose tokens that a larger target model verifies in parallel; its benefit depends on whether the reduction in target-model calls compensates for drafting, verification and cache-management costs. On an 8GB consumer GPU, those costs must be measured alongside the memory needed to keep both models resident.

This project asks one primary research question: **How does context length affect the decoding speedup of greedy speculative decoding relative to autoregressive decoding on an 8GB consumer GPU?** We compare approximately 2K, 4K and 8K contexts while fixing the model pair, uncompressed BF16 KV cache, proposal length and validated configuration. Acceptance statistics and component timings will explain the measurements. A prespecified 8K V-only FP8 ablation will check sensitivity to cache representation at the longest context; it will not estimate a compression-dependent break-even curve.

We assume neither that longer contexts reduce speedup nor that speculative decoding is faster at any tested length; speedups below, near or above $1\times$, or the absence of a break-even point in range, are all reportable. This is a controlled empirical evaluation of one implementation and hardware setting, not a new decoding algorithm, quantization method or inference engine.

2 Related Work and Baselines

Leviathan et al. introduce speculative decoding using draft proposals and target verification [1]. We study a fixed-chain greedy implementation with Qwen2.5 models [2], focusing on how its measured benefit varies with context length. WikiText supplies reproducible text for the paired prompt families [3].

The primary baseline is cached AR decoding with **only the target model resident**. Speculative decoding uses the same target weights, tokenizer, input, precision and attention policy while paying for its necessary draft model; Hugging Face Transformers eager is an external correctness reference, not an extra condition. Results are scoped to the tested model pair, implementation and GPU.

Completed Preliminary Work

The project already has a working inference platform and diagnostic tools (forward computation, KV caching, greedy generation and chunked prefill, plus fixed-chain speculative decoding, cache trimming and repair), reused below for six main conditions and two supporting 8K conditions. Target-model reference and cache checks passed in their recorded settings, and 8K inference was demonstrated operationally; the results are preliminary evidence from recorded development configurations, not results from the proposed benchmark.

Correctness evidence, split into two layers. We separate *control-logic* correctness from the *numerical behaviour* of batched verification. A serial verification oracle runs target verification with the same per-token execution as AR, reusing the production accept, replacement, rollback and cache-commit logic verbatim under a call-geometry audit that rules out multi-token verification forwards. On five development and protocol-validation families (F13, F01, F02, F08, F10) it reproduced the target-only AR token IDs exactly (64/64) under BF16 KV, with independent repetitions and structural checks. This supports that control logic *on those inputs* only; it is not a proof for all inputs and covers `base_bf16` alone. Two earlier records remain in force: row-aligned failed the algorithmic comparison on three of four protocol-validation families, and the earlier BF16 gate set recorded one unresolved divergence (19/20). Production BF16 batched verification still differs from AR on some inputs, and two points must not be merged. Shape sensitivity in five operator classes (attention QK and the four projections) was confirmed by causal experiments on specific same-history samples; this identifies a mechanism but does not account for every production divergence, so we claim no operator-level localisation for all of them. The five-family serial result shows that complete per-token target verification recovers the AR output on those inputs; neither point establishes control logic for all inputs. The oracle is diagnostic only; its timing is excluded from all performance results. The formal correctness gate, the pilot and the main matrix are therefore **not** passed by this evidence.

Completed FP8 exploration. We implemented asymmetric V-only FP8 storage: K remains BF16 while V uses E4M3 with a per-token, per-KV-head FP32 max-absolute scale (group size 128), dequantized for eager attention; this is compressed storage, not native FP8 attention. Full K/V and K-only FP8 variants were also explored and degraded quality substantially in the tested setting.

Table 1. Recorded V-only FP8 preliminary findings

Item	Observed result and scope
Quality protocol	WikiText-2 test; 40 fixed non-overlapping 2048-token windows, disjoint from the quantization pilot; chunk 512.
Quality change	Mean NLL increase approximately 0.000397; relative PPL increase approximately 0.040% against uncompressed BF16 KV.
Logical target KV at 8K	224.00 MiB to 169.75 MiB, including scale metadata: approximately 1.32× compression. Not whole-model compression.
Paired operational test	At 8K, chunk 2048, 64 generated tokens: allocated-memory peak decreased by approximately 0.06 GiB; decode was not faster in this eager test.
Evidence boundary	Qwen2.5-1.5B, BF16 compute, tested eager implementation. These numbers measure quality under a changed KV representation only; they cannot by themselves show that AR and production speculative decoding generate equally good text after diverging, and they do not transfer to another backend or the formal configuration.

These findings are retained in the final report, including the negative K-FP8 results and their configuration limits. The main study fixes BF16 KV; a bounded follow-up reuses the V-only implementation for an AR/Spec comparison at 8K, selected in advance as the longest planned context rather than from observed speedups. This preserves the completed work without reinstating the original 12-condition factorial or adding new formats or sweeps.

Measurement infrastructure. Existing tooling records configurations, token outputs, failures, allocator and device-memory statistics and environment observations, and already provides independent-process runs, per-round control traces, call-geometry audits and divergence records, plus separate instrumentation passes. The formal experiment will enable and verify these fields under the frozen protocol; formal timing still requires pilot validation.

Remaining work. Complete the control-logic correctness gate for the second KV representation and the final-configuration regression, validate the six main conditions, then run the two supporting conditions if their gates pass, and interpret the context-length results. We will report preliminary evidence separately from the formal measurements and will not pool incompatible configurations.

3 Proposed Method

The existing PyTorch engine supports cached greedy generation and chunked prefill. Fixed-chain Spec proposes up to four tokens, accepts the prefix matching target decisions and repairs a mismatch using the target; cache synchronization and rollback maintain the committed prefix.

The attention policy must pass correctness and operational validation, then remain fixed across all conditions. AR and Spec use the same target prefill and phase-specific policy. Our correctness gate is layered and applies separately within each KV representation. *Control logic*: with the serial verification oracle, the complete generated token IDs must match target-only AR at the same KV representation, alongside cache and per-round control invariants, finiteness, determinism and legal stopping. *Production numerical behaviour*: verification keeps the original batched path, and the admission rules below decide what is admissible. `base_bf16` and `v_fp8` are each verified within their own representation; identical tokens across the two representations are not required, and the five-family result above covers `base_bf16` only, so the FP8 condition still needs its own control-logic verification. Admission rules are explicit. Non-finite values, cache or per-round structural failures, and non-reproducible repetitions are hard failures that stop the condition. A token divergence between production and AR must be recorded and explained in full: first position, common prefix, full outputs, acceptance and work volume. Such a divergence is never automatically treated as harmless, and it does not support any claim of lossless speedup at identical output.

Table 2. Shared configuration and the single supporting variation

Setting	Fixed value or rule
Hardware and compute	RTX 4060 Laptop GPU, 8GB VRAM, 32GB system RAM; BF16 compute; target KV per Table 3; draft KV always uncompressed BF16
Target model	Qwen2.5-1.5B-Instruct
Draft model (Spec only)	Qwen2.5-0.5B-Instruct
Decoding	Greedy argmax; fixed-chain proposal length $k = 4$
Prefill	Chunk size 2048; identical target prefill policy in AR and Spec
Attention policy	One validated, frozen phase-specific policy; no backend sweep
Generation limit	At most 64 new tokens; stop on the frozen tokenizer EOS; record actual length and stop reason
Baseline residency	Target-only for AR; target and draft for Spec

Table 3. Six main conditions plus two prespecified 8K ablations

ID	Context	Decoding method	Target KV
C1	2K	Autoregressive (AR)	BF16
C2	2K	Speculative ($k = 4$)	BF16
C3	4K	Autoregressive (AR)	BF16
C4	4K	Speculative ($k = 4$)	BF16
C5	8K	Autoregressive (AR)	BF16
C6	8K	Speculative ($k = 4$)	BF16
A1	8K	Autoregressive (AR)	V-only FP8
A2	8K	Speculative ($k = 4$)	V-only FP8

C1/C2, C3/C4 and C5/C6 form the main matched pairs, with speedup as AR time divided by Spec time. A1/A2 form a separate same-KV pair (target K stays BF16, V uses the existing FP8 scheme); its 8K speedup and memory are compared with C5/C6 without inferring a compression-by-length interaction. No FP8 conditions at 2K/4K, weight quantization, custom CUDA kernels or model/k tuning are planned.

4 Data, Models, Setup, and Evaluation

Inputs and pilot. Prompt families use WikiText-2 training-split text [3], with source locations, tokenizer settings and token hashes recorded in a fixed manifest. Within a family, 2K, 4K and 8K inputs are nested prefixes of the same base text with the same continuation task. Actual token counts are recorded (tolerance at most 32 tokens), with no synthetic filler. A four-family pilot checks correctness, resources, instrumentation and timing stability before at least 16 formal prompt families are measured. Previously inspected families remain development inputs, not held-out validation evidence. Public-corpus overlap with model pretraining is possible; the study measures paired system behavior, not uncontaminated generalization.

Measurement protocol. The supporting pair uses the same formal families' 8K prefixes and repetition protocol as C5/C6. We retain one warm-up and five measured repetitions per condition, distributed across at least two randomized blocks. Process isolation and a consistent environment check limit carry-over effects. Code revisions, configurations, output lengths, EOS behavior, clock/temperature observations and failures are recorded. Instrumented cost breakdowns are collected separately from primary timing runs. Repetitions are summarized by their median within each family; paired family-level results are then summarized by median and interquartile range, rather than selecting the fastest run.

Table 4. Evaluation measures and interpretation

Measure	Purpose
Decode speedup (primary)	$S(L) = T_{\mathrm{AR,decode}}(L)/T_{\mathrm{Spec,decode}}(L)$ under matched inputs, KV representation, generation limit and timing boundary; $S > 1$ means faster decoding. Where the two paths generate different tokens, the ratio is reported as a comparison *across divergent trajectories*, with divergence rate, first-divergence distribution, actual lengths and acceptance/round/target-call counts; no lossless same-output claim is made. Diagnostic (serial or row-aligned) runs are never used to compute speedup.
Latency and throughput	End-to-end seconds (target prefill for AR; target plus draft prefill for Spec) and generated tokens/s, each with actual output length and the timing boundary recorded.
Acceptance behavior	Acceptance rate and accepted tokens per verification help explain target-call savings; report each trajectory's work volume separately once outputs diverge.
Component timings and memory	Separately measured draft and verification latency, logical KV bytes, allocated/reserved peaks and device occupancy; allocator statistics are not total physical residency.
Correctness and failures	Full generated token IDs, length, and stop reason; finite outputs and cache invariants; per-round control-invariant and control-logic gate status; recorded AR-versus-Spec divergence with first position and common prefix; OOM, crashes, and blocked conditions retained.

Analysis and feasibility

Main conclusions use C1–C6; a crossing of 1× yields an interval, not an extra length sweep. A1/A2 are secondary, gated by same-KV correctness and stable execution. If the adopted backend differs from the recorded eager configuration, repeat the existing 40-window quality check and report it separately; old quality numbers are not transferable by assumption. Blocked ablations do not prevent a validated main study. No training or paid APIs are needed; remaining effort and GPU hours will be recalibrated from the pilot against the original 140–175 person-hour estimate. All blocked cells and mismatched trajectories remain visible, without equivalent-output speedup claims.

5 Timeline and Roles

Week	Milestone	Owner(s)
8–9	Finish the control-logic correctness gate for both KV representations, freeze the entry-point configuration, and validate data provenance.	Fang Chuming; Deng Xiyue; Cheng Shuyue
10–11	Freeze the validated configuration; run the four-family pilot, then the six main conditions and gated 8K ablation, recording output divergence for each pair.	All members
12–13	Analyze context-length effects; prepare figures and the report draft; audit reproducibility; finalize the report and video; rehearse the presentation.	Cheng Shuyue; all members

Fang Chuming owns inference-engine maintenance, target/KV correctness and regression including the per-representation control-logic gate, chunked-prefill integration, configuration consistency, memory profiling and V-only FP8 regression for the supporting pair. **Deng Xiyue** owns speculative decoding, cache synchronization and rollback, control-logic output-equivalence validation through the serial oracle, production-divergence recordkeeping, acceptance instrumentation and latency measurement. **Cheng Shuyue** owns paired prompt families, experiment orchestration, results-schema validation, repeated-run checks, external-reference coordination, aggregation and figures. All members review correctness evidence, reproducibility and interpretation, and distinguish exploratory results from new formal measurements.

6 Responsible Use and Tool Disclosure

We will follow the models' Apache-2.0 terms and WikiText's attribution/share-alike terms, with sources and licenses recorded [4, 5]. We use public text only, and will not present generated content as verified fact. AI assistants support coding, debugging and writing, subject to course policy; their use will be disclosed. Team members remain responsible for verifying code, citations, experiments and analysis; raw failures and negative results are retained.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML, 2023. arXiv:2211.17192.

[2] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv:2412.15115.

[3] S. Merity, C. Xiong, J. Bradbury, and R. Socher. *Pointer Sentinel Mixture Models*. 2016. arXiv:1609.07843.

[4] Qwen Team. Official model cards and licenses: Qwen2.5-1.5B-Instruct and Qwen2.5-0.5B-Instruct.

[5] Salesforce. *WikiText Dataset Card*. huggingface.co/datasets/Salesforce/wikitext.